



TECHNICAL REFERENCE

TP Local File Generator

AI-assisted authoring system for Indonesian Transfer Pricing documentation under **PMK-213/2016**.

DOCUMENT	VERSION	ISSUED	OWNER
README · Engineering Reference	1.0	April 2026	RSM Indonesia · Tax Technology

OVERVIEW

What this is

An AI-powered web application that automates the creation of **Transfer Pricing (TP) Local File** documentation — the mandatory compliance report required by Indonesian tax regulations `(PMK-213/2016)` for companies conducting affiliated transactions above the reporting threshold.

Traditionally a tax consultant spends **weeks** manually researching, writing, and formatting this document. This application reduces that to **hours** by combining a structured data-entry wizard with an AI agent pipeline that researches, writes, and formats each section automatically — then exports the result as a ready-to-use `.docx` file.

Why "TP Local File"?

For developers unfamiliar with Indonesian tax regulations, the name may seem unusual. **Transfer Pricing (TP)** refers to the pricing of transactions between related parties (affiliates) — for example, between a parent company and its subsidiaries, or between sister companies within the same multinational group.

Under PMK-213/2016, Indonesian companies that conduct affiliated transactions above a certain threshold are legally required to prepare three transfer pricing documents:

#	Document	Indonesian Name	Contents
1	Master File	<i>Dokumen Induk</i>	Overview of the multinational group: global structure, business, and group-wide TP policies
2	Local File	<i>Dokumen Lokal</i>	Details of the Indonesian entity's affiliated transactions: functional analysis, TP method, financial data
3	CbCR	<i>Laporan per Negara</i>	Aggregated financial data by country (only for groups with revenue > IDR 11 trillion)

This application specifically generates document **#2 — the Local File (*Dokumen Lokal*)**. The name was chosen deliberately so that tax consultants — the primary users — immediately understand which document the tool produces, without ambiguity.

New developer? Read these in order

1. Architecture (p.4) — the five services and how they talk. **2.** Directory Structure (p.11) — where everything lives; `tp_app/` is the AI brain, `backend/` is the API, `frontend/` is the wizard. **3.** Quick Start (p.7) — get it running locally in ~10 minutes. **4.** Agent Pipeline (p.15) — the LangGraph that produces the document. Skim everything else; come back when you need it.

Table of Contents

01	Architecture Overview	04
Service map, why the design works		
02	Tech Stack	05
Frontend, backend, AI & agents		
03	Quick Start	07
Prerequisites, API keys, 9-step setup		
04	Environment Variables	11
.env reference		
05	Directory Structure	12
Annotated repo layout		
06	User Flow & Authentication	13
Wizard journey, JWT lifecycle		
07	Document Retrieval — PageIndex	14
Two-tier strategy, RAG comparison		
08	AI Agent Pipeline	15
LangGraph parallel orchestration		
09	API Reference	16
Auth, projects, tasks, admin		
10	CLAUDE.md & Graphify	17
AI assistant rules, knowledge graph		
11	Development Guide	18
Docker, local, agent testing, git		
12	Deployment	19
CI/CD, GitHub Secrets, EC2 server		
<i>tp-local-file-generator</i>		03

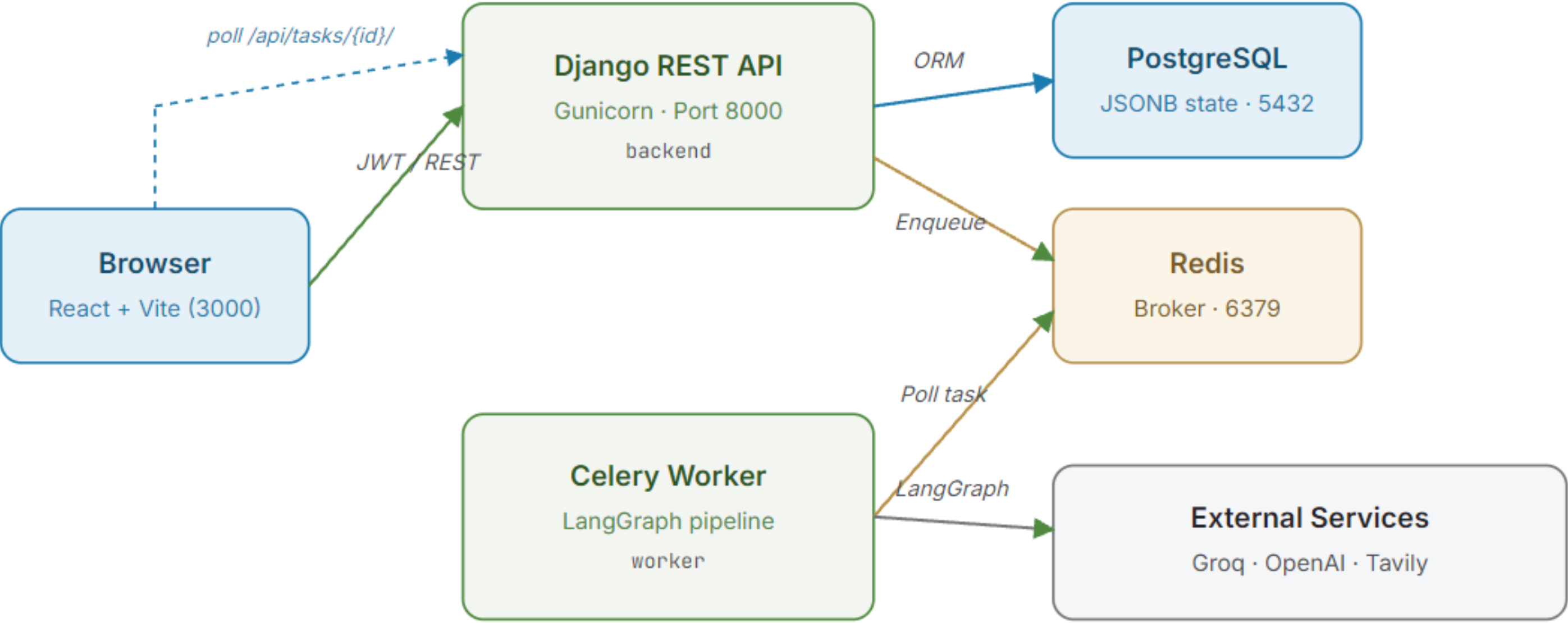
SYSTEM

Architecture Overview

The application is a five-service Docker stack: a React frontend, a Django REST API, a Celery worker for async AI tasks, a PostgreSQL database, and a Redis broker. AI work runs out-of-process so the request thread stays responsive.

Mental model in one paragraph

The user fills out a wizard in the **browser**. Wizard data is sent to the **backend** and saved into a single JSONB column on a `Project` row in **Postgres**. When the user hits "Run AI Agents", the backend creates an `AgentTask` row, pushes a job onto **Redis**, and returns a task ID. The **worker** picks the job up, runs the `tp_app/` LangGraph pipeline (calling Groq/OpenAI/Tavily), writes results back into the same JSONB state, and updates the `AgentTask` with progress. The frontend polls `/api/tasks/{id}/` every few seconds to show progress. When done, the user clicks "Export DOCX" and the backend renders the JSONB state into a Word file.



Five-service stack — Celery offloads multi-minute LLM pipelines so the request thread stays responsive.

Services

Service	Image	Port	Purpose
db	postgres:16-alpine	5432	Primary database
redis	redis:7-alpine	6379	Celery broker & result backend
backend	Python 3.11-slim	8000	Django REST API (Gunicorn)
worker	Python 3.11-slim	—	Celery async worker (AI tasks)
frontend	Node 20-alpine	3000	React dev server (Vite)

Why this design?

- **JSONB state storage** — The TP Local File state is a deeply nested, schema-evolving document. Storing it as a JSONB column avoids schema migrations every time a new field is added, and makes import/export trivial.
- **Celery + Redis for async AI tasks** — LLM pipelines take 2–5 minutes. Celery offloads these to background workers while the frontend polls `/api/tasks/{id}/` for live progress.
- **Volume-mounted `tp_app/`** — Agent code, templates, and export logic are mounted into **both** the backend and worker containers. The backend imports it for synchronous things (e.g. DOCX export); the worker imports it for the async pipeline. Code edits take effect without rebuilding Docker images during development.
- **LangGraph parallel orchestration** — Independent TP sections run in parallel branches, reducing total generation time by ~60% vs. sequential execution.

QUICK REFERENCE

Where to start coding

If you've just cloned the repo and want to ship something useful in your first afternoon, this is the lookup table. Each row maps an intent to the exact file (or pair of files) you'll need to open.

I want to...	Open this file
Add or change a wizard step	<code>frontend/src/pages/Step*.tsx</code>
Add a new API endpoint	<code>backend/api/views.py</code> + <code>backend/api/urls.py</code>
Add a field to project state	Just write to it — JSONB has no schema. Update the TS type in <code>frontend/src/types/</code> .
Add or modify an AI agent	<code>tp_app/agents/*_subagent.py</code>
Change the LangGraph topology	<code>tp_app/agents/orchestrator.py</code>
Change the DOCX output	<code>tp_app/TP_Local_File_TEMPLATE.docx</code> (Jinja2) or <code>tp_app/export/docx_export.py</code> (programmatic)
Change the LLM provider	<code>tp_app/agents/llm_factory.py</code>

Tip · Read the agent before you write one

Each subagent file in `tp_app/agents/` follows the same pattern: a Pydantic input schema, a prompt template, an LLM call, and a structured output. Open `industry_subagent.py` first — it's the simplest one — then duplicate it as a starting point for new agents.

INVENTORY

Tech Stack

Three layers — a typed React frontend, a Django REST backend, and an AI-agent module living in `tp_app/` that both backend and worker mount.

Frontend

Technology	Version	Purpose
React	18	Component-based wizard UI
TypeScript	5	Type safety across components & API contracts
Vite	5	Dev server and production bundler
Tailwind CSS	3	Utility-first styling
Zustand	5	Global state (wizard data, auth, settings)
React Router DOM	6	Client-side routing
TanStack Query	5	Server state caching & background refetch
Axios	1	HTTP client for REST API calls
React Dropzone	14	Drag-and-drop file upload
Radix UI	latest	Accessible headless UI primitives

Backend

Technology	Version	Purpose
Python	3.11	Primary language
Django	5.0.6	Web framework, ORM, admin
Django REST Framework	3.15.2	REST API serializers & views
<code>django-rest-framework-simplejwt</code>	5.3.1	JWT authentication
Celery	5.4.0	Async task queue
Redis / kombu	5.0.8	Message broker for Celery
<code>psycopg2-binary</code>	2.9.9	PostgreSQL adapter
Gunicorn	22.0.0	WSGI production server
WhiteNoise	6.7.0	Static file serving
<code>python-docx</code>	1.1.2	Programmatic DOCX generation
<code>docxtpl</code>	0.18.0	Jinja2 templating over Word files

AI & Agents (tp_app/)

Technology	Purpose
LangGraph	Stateful parallel AI agent orchestration
LangChain	LLM abstraction, prompt management
Groq default	Fast LLM inference (llama-3.3-70b-versatile)
OpenAI optional	Alternative LLM provider (GPT-4o etc.)
Tavily	AI-optimized web search for industry research
PageIndex	Vectorless hierarchical PDF retrieval
FAISS	In-memory vector store (Vector RAG fallback)
sentence-transformers	HuggingFace local embeddings (free)
PyPDF2 / pymupdf	PDF parsing
tiktoken	Token counting

Two retrieval strategies

The AI module ships with both a **vectorless PageIndex** traversal for short, well-structured PDFs and a classic **FAISS Vector RAG** fallback for longer or unstructured documents. The router in document_processor.py chooses automatically based on page count and which API keys are configured. See the retrieval chapter on page 14.

SETUP

Quick Start

Prerequisites

Tool	Version	Install
Docker & Docker Compose	latest	docs.docker.com/get-docker
Python	≥ 3.11	python.org/downloads
Node.js	≥ 20	nodejs.org
uv	latest	see Step 1
Git	latest	git-scm.com

Plus at least one API key from each: **LLM** (Groq free tier or OpenAI) and **Web search** (Tavily).

API Keys — How to Get Them

Groq recommended · free tier

1. Go to console.groq.com
2. Click **Sign Up** — register with Google or email
3. After login, go to **API Keys** in the left sidebar
4. Click **Create API Key** → name it (e.g. `tp-app`)
5. Copy the key — starts with `gsk_ ...`
6. Paste it into **Admin Settings** in the app (not `.env`)

Free tier rate limits are generous enough for development and light production use.

OpenAI optional · paid

1. Go to platform.openai.com
2. Click **Sign Up** or **Log In**
3. Go to [API Keys](#)
4. Click **Create new secret key** → name it
5. Copy the key — starts with `sk- ...`
6. Make sure your account has billing enabled (add a credit card under Billing)
7. Paste it into **Admin Settings** in the app

Recommended models: `gpt-4o-mini` for cost efficiency, `gpt-4o` for higher quality.

Tavily required for AI research

Tavily is used by the AI agents to search the web for industry analysis, Indonesian regulations, and comparable company data.

1. Go to app.tavily.com
2. Click **Sign Up** — register with Google or email
3. After login, your API key is shown on the dashboard immediately
4. Copy the key — starts with `tvly- ...`
5. Paste it into **Admin Settings** in the app

Free tier includes 1,000 searches / month. Each full AI pipeline run uses ~5–15 Tavily searches.

Step 1 — Install `uv` (Python package manager)

`uv` is required to manage all Python dependencies in this project. Install it once globally:

macOS / Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Verify:

```
uv --version
```

Step 2 — Clone the repository

```
git clone <repo-url>
cd tp_local_file_generator
```

Step 3 — Configure environment variables

```
cp .env.example .env
```

Open `.env` and fill in the required values (see the Environment Variables chapter for reference). At minimum set `SECRET_KEY` and `POSTGRES_PASSWORD`.

Step 4 — Install Python dependencies (tp_app)

tp_app/ is the AI agent module. Install its dependencies with uv :

```
cd tp_app
uv sync
cd ..
```

This reads tp_app/pyproject.toml and installs everything into an isolated .venv inside tp_app/.

Step 5 — Install frontend dependencies

```
cd frontend
npm install
cd ..
```

Step 6 — Build and start all services

```
docker compose up --build
```

This builds and starts: db (PostgreSQL), redis , backend (Django), worker (Celery), frontend (React/Vite).

First build takes ~3–5 minutes
Subsequent starts with docker compose up -d are fast.

Step 7 — Run database migrations & create superuser (first time only)

Wait for the backend to be healthy (watch logs with docker compose logs -f backend), then:

```
docker compose exec backend python manage.py migrate
docker compose exec backend python manage.py createsuperuser
```

Step 8 — Open the app

Service	URL
Frontend (app)	http://localhost:3000
Django API	http://localhost:8000/api/
Django Admin	http://localhost:8000/admin/

Step 9 — Configure API keys

Log in as the superuser you just created → **Admin Settings** → enter:

- LLM Provider (Groq or OpenAI)
- LLM API Key
- Tavily API Key

These are stored in the database and shared across all users — no need to put them in .env .

Quick reference (after initial setup)

```
# Start all services (background)
docker compose up -d

# Stop all services
docker compose down

# Rebuild after code changes
docker compose up -d --build backend worker

# View backend logs
docker compose logs -f backend

# View worker (AI agent) logs
docker compose logs -f worker

# Run Django management commands
docker compose exec backend python manage.py <command>
```

Environment Variables

Create `.env` at the project root (copy from `.env.example`):

```
# — Django —
SECRET_KEY=change-me-to-a-long-random-string
DEBUG=True
ALLOWED_HOSTS=*

# — PostgreSQL —
POSTGRES_DB=tp_db
POSTGRES_USER=tp_user
POSTGRES_PASSWORD=tp_pass

# — Redis (Celery) —
REDIS_URL=redis://redis:6379/0
```

API keys live in the database

LLM and Tavily API keys are **not** in `.env`. Configure them from the Admin Settings page after first login.

REPOSITORY

Directory Structure

```

tp_local_file_generator/
|
├── frontend/                                React + TypeScript + Vite
│   └── src/
│       ├── pages/                          LandingPage, LoginPage, ProjectDashboard,
│       │                               Step0Upload ... Step11ReviewExport
│       ├── components/                    Sidebar, SettingsModal, DynamicTable, etc.
│       ├── store/                        Zustand: projectStore, authStore
│       ├── api/                          Axios client (projects, auth, tasks)
│       └── types/                         TypeScript interfaces
│
├── backend/                                Django REST API
│   ├── config/                            settings.py, urls.py, celery.py, wsgi.py
│   └── api/
│       ├── models.py                     Project (JSONB), AgentTask, SystemSetting
│       ├── views.py                      All API endpoints
│       ├── tasks.py                      Celery async tasks
│       ├── serializers.py
│       └── services/
│           └── agent_service.py           Django ↔ LangGraph bridge
│
├── tp_app/                                AI agents + DOCX export (volume-mounted)
│   ├── agents/
│   │   ├── orchestrator.py               LangGraph StateGraph (parallel pipeline)
│   │   ├── research_subagent.py          Tavily web research nodes
│   │   ├── analysis_subagent.py          Functional analysis + characterization
│   │   ├── transaction_subagent.py       Comparability, method, PLI justification
│   │   ├── summary_subagent.py           Executive summary, conclusion, P/L overview
│   │   ├── business_subagent.py          Business activities, supply chain
│   │   ├── extraction_agent.py           Document extraction (PageIndex / Vector RAG)
│   │   └── llm_factory.py                LLM provider factory (Groq / OpenAI)
│   ├── export/
│   │   ├── docx_export.py                python-docx builder export
│   │   └── docx_template_export.py       docxtpl Jinja2 template export
│   ├── utils/
│   │   └── document_processor.py         2-tier retrieval strategy
│   └── TP_Local_File_TEMPLATE.docx       Pre-formatted Jinja2 Word template
│
├── graphify-out/                          Codebase knowledge graph (auto-generated)
│   ├── GRAPH_REPORT.md                   God nodes + community structure summary
│   └── graph.json                        Full graph data
│
├── .github/workflows/
│   └── ci-cd.yml                         GitHub Actions: test + deploy
│
├── docker-compose.yml
├── .env                                  (gitignored – create from .env.example)
├── CLAUDE.md                             AI assistant project instructions
└── README.md

```

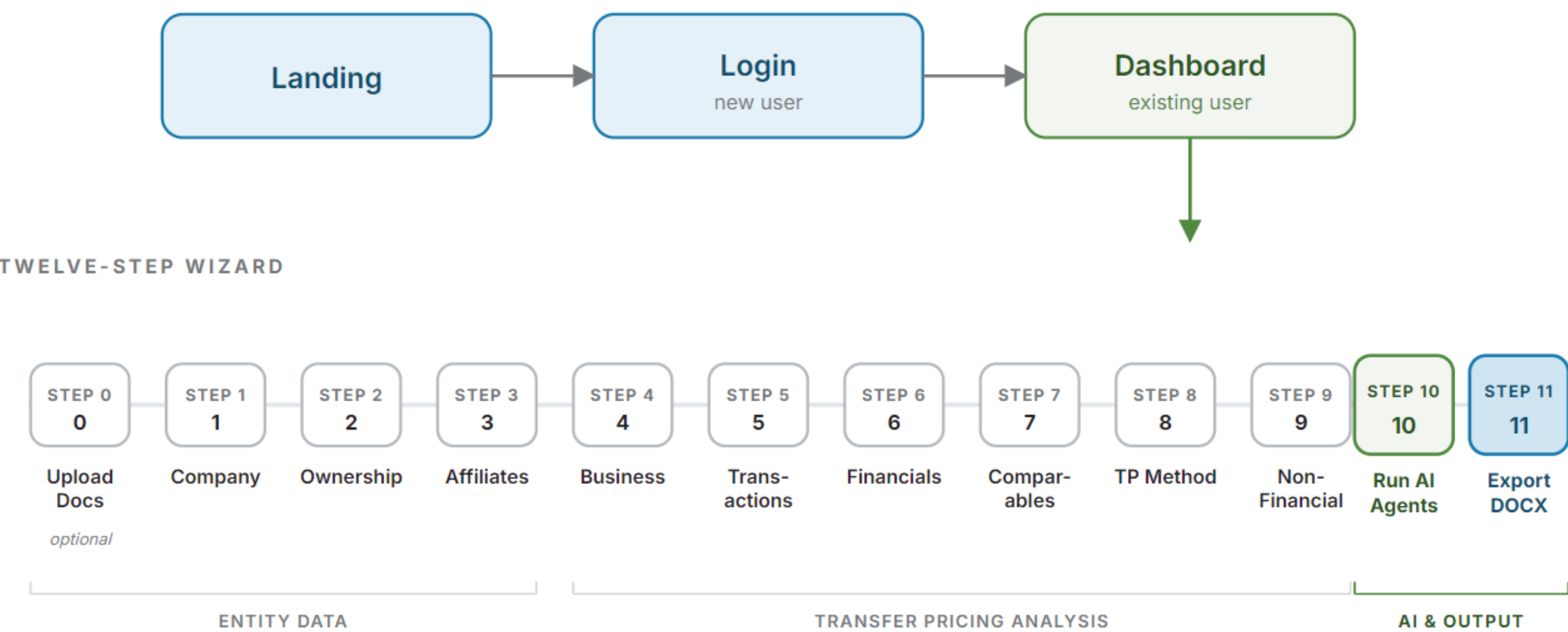
WIZARD JOURNEY

User Flow

The wizard takes a consultant from a fresh login to an exported `.docx` in twelve linear steps. Existing projects re-enter at the dashboard.

Step 0 is optional but recommended

If the consultant uploads a prior year's TP document, the AI extracts company name, shareholders, financials etc. and pre-fills steps 1–9. If they skip it, every field is filled in manually. Step 0 never blocks progress.



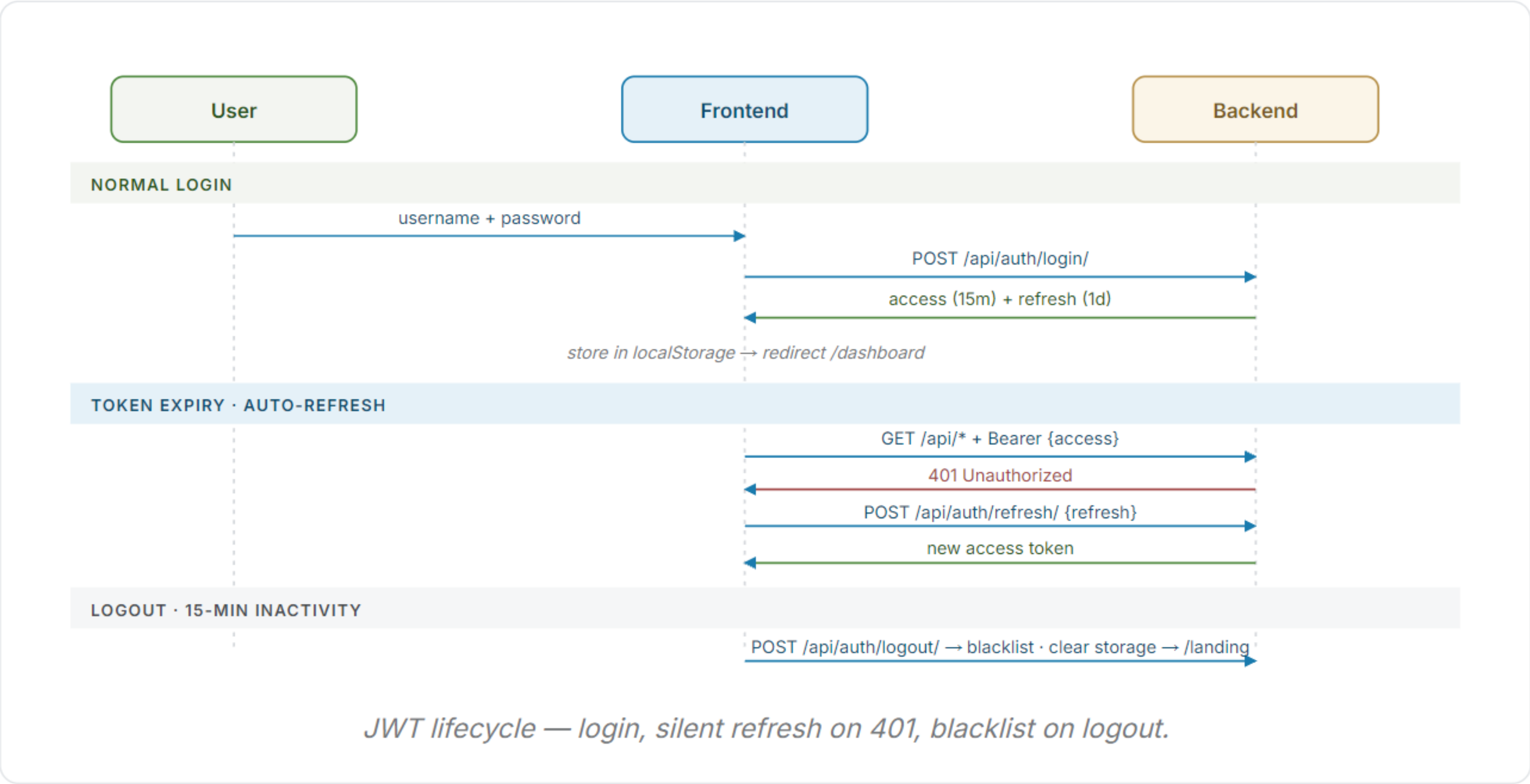
Twelve-step wizard, single linear flow. Steps 0–3 capture entity data; 4–9 build the TP analysis; Step 10 hands off to the LangGraph pipeline; Step 11 exports the finished DOCX.

Twelve-step wizard. Step 10 hands off to the LangGraph pipeline; Step 11 exports the finished DOCX.

SECURITY

Login & Authentication

Authentication uses JWT (JSON Web Tokens) via `django-rest-framework-simplejwt`. Tokens live in `localStorage` on the client; refresh rotation and blacklisting are enabled.



Token settings

Access token lifetime	15 minutes
Refresh token lifetime	1 day
Refresh rotation	Enabled
Token blacklisting	Enabled
Inactivity auto-logout	15 minutes

Admin vs regular user

- **Regular users** — full access to wizard, project management, and export
- **Admin** (`is_staff=true`) — additionally can manage users and configure global API keys via the Admin Settings page

EXTRACTION

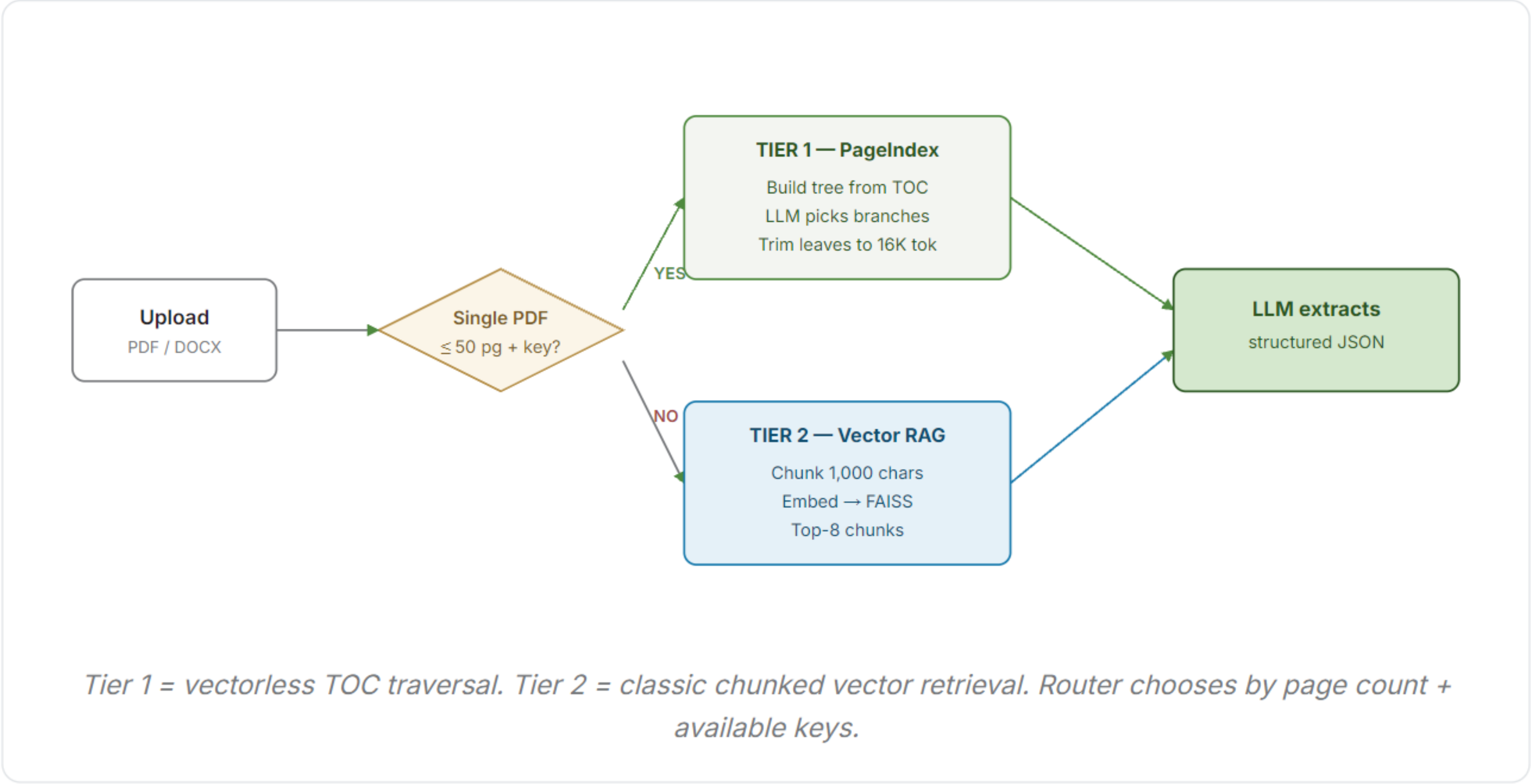
Document Retrieval — PageIndex

When a user uploads a prior TP document in Step 0, the system extracts structured data (company name, shareholders, financials, etc.) using a **2-tier retrieval strategy** defined in `tp_app/utils/document_processor.py`.

Router decision

Condition	Tier used
Single PDF, ≤ 50 pages, OpenAI key set	Tier 1 — PageIndex
PDF > 50 pages	Tier 2 — Vector RAG
Not a PDF (DOCX, scanned image, etc.)	Tier 2 — Vector RAG
Only a Groq key configured	Tier 2 — Vector RAG

Tier 1 needs a model that can reason over a long TOC tree in one pass; today that's OpenAI GPT-4o. Tier 2 is the safe default — it works with any embeddings model.



Tier 1 — PageIndex (vectorless, for PDFs ≤ 50 pages)

PageIndex builds a hierarchical tree from the document's table of contents, then lets the LLM navigate level by level — reading only summaries at each level until it reaches the relevant leaf nodes. Full text is only read at the final selected nodes, mimicking how a human expert skips irrelevant chapters.

Comparison with traditional RAG

Aspect	Vector RAG	PageIndex
Search method	Embedding cosine similarity	LLM reasoning over tree structure
Storage needed	Vector database	None
Retrieval style	One-shot similarity match	Multi-level hierarchical traversal
Best for	Long / unstructured docs	Structured docs with TOC (≤ 50 pg)

Tier 2 — Vector RAG (long or non-PDF documents)

Text is split into 1,000-character chunks (150-char overlap), embedded using HuggingFace `all-MiniLM-L6-v2` (local, free) or OpenAI `text-embedding-3-small`, stored in FAISS, and top-8 chunks are retrieved per query.

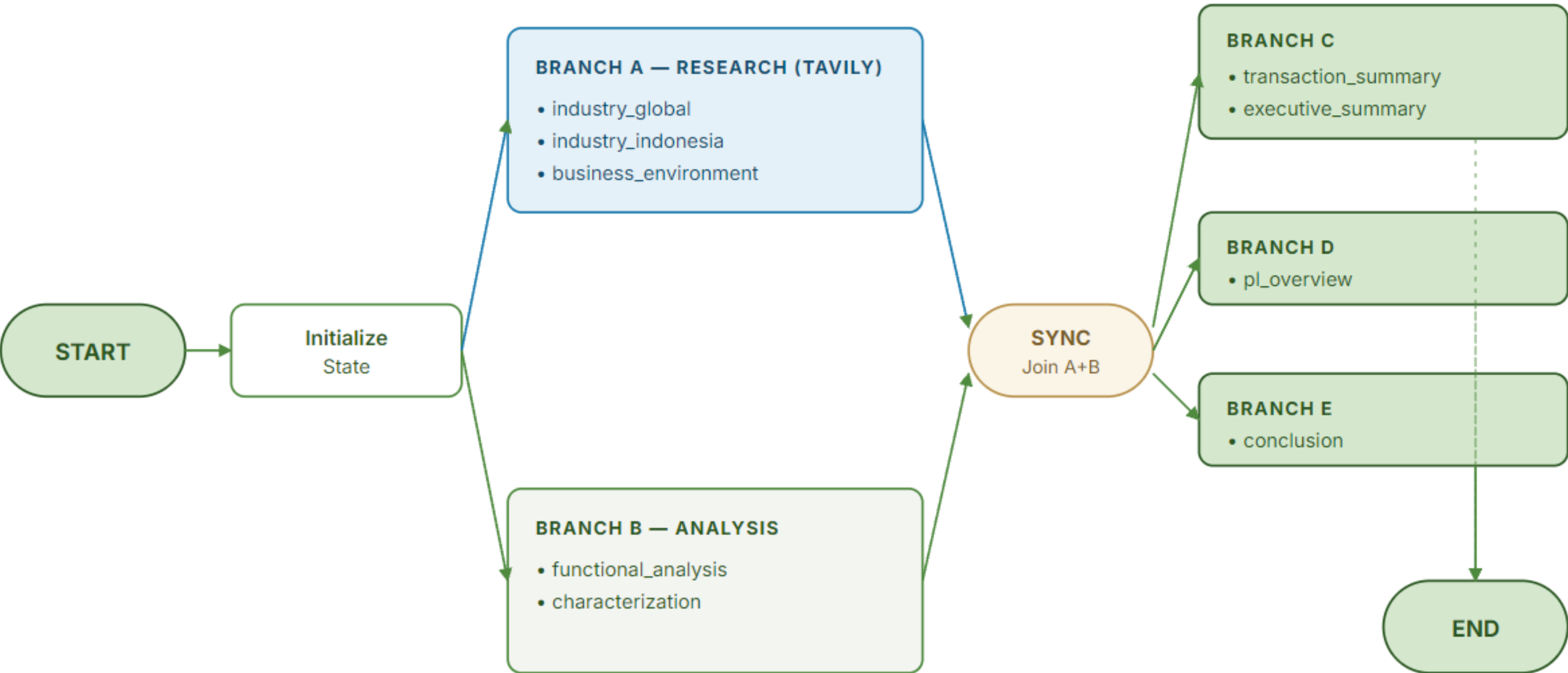
GENERATION

AI Agent Pipeline

The AI generation pipeline in `tp_app/agents/orchestrator.py` uses a LangGraph `StateGraph` for parallel execution. Branches A and B run in parallel, then C, D and E fan out after a sync barrier.

Async task lifecycle (end-to-end)

- 1. Frontend POSTs `/api/projects/{id}/run-agents/`.
- 2. Backend creates an `AgentTask` row, enqueues a Celery job with the task ID, returns `{ task_id }`.
- 3. Worker dequeues, loads project state from JSONB, runs `tp_app/agents/orchestrator.py`, writing each section back to JSONB as it completes.
- 4. Each completed node updates the `AgentTask.progress_log`.
- 5. Frontend polls `GET /api/tasks/{id}/` every 2s to render a live progress UI.
- 6. When status flips to `SUCCESS`, the frontend re-fetches the project and shows Step 11.



~60% faster than sequential — A & B run in parallel; C, D, E fan out after the sync barrier.

Supported LLM providers

Provider	Model
Groq	llama-3.3-70b-versatile
OpenAI	gpt-4o, gpt-4o-mini

DOCX export paths

Type	Engine
?type=template	docxtpl + Jinja2
?type=builder	python-docx

REST

API Reference

Authentication

Method	Endpoint	Description
POST	/api/auth/login/	Login → returns JWT tokens
POST	/api/auth/logout/	Logout → blacklist refresh token
POST	/api/auth/refresh/	Refresh access token
GET	/api/auth/me/	Current authenticated user info

Projects

Method	Endpoint	Description
GET	/api/projects/	List all projects
POST	/api/projects/	Create new project
GET	/api/projects/{id}/	Get full project state
PATCH	/api/projects/{id}/	Save state updates
DELETE	/api/projects/{id}/	Delete project
GET	/api/projects/{id}/export-json/	Download project as JSON backup
POST	/api/projects/{id}/load-json/	Restore from JSON backup
POST	/api/projects/{id}/upload-documents/	Upload docs → async AI extraction
POST	/api/projects/{id}/run-agents/	Run full AI pipeline (async)
POST	/api/projects/{id}/run-single-agent/	Regenerate one section (async)
GET	/api/projects/{id}/export-docx/?type=template	DOCX (docxtpl)
GET	/api/projects/{id}/export-docx/?type=builder	DOCX (python-docx)

Tasks & Config

GET	/api/tasks/{id}/
GET	/api/config/

Admin

GET/PATCH	/api/admin/settings/
GET/PATCH	/api/admin/users/

ASSISTANT RULES

CLAUDE.md — AI Assistant Instructions

This project includes `CLAUDE.md` at the root (and `.claude/CLAUDE.md`) instructing Claude or any AI coding assistant on how to work with this codebase. Read this file before starting development with an AI assistant.

Key rules

Rule	Reason
Always use <code>uv</code> for Python packages	Consistent lockfile, faster than pip
Run <code>docker ps</code> before debugging	Confirm the right containers are running
Use <code>docker logs <id></code> for debugging	Inspect container output
Use <code>docker compose up -d --build</code> after changes	Rebuild and restart cleanly
Always commit after updating code	Prevents lost work
Read <code>graphify-out/GRAPH_REPORT.md</code> first	Cross-module relationships not visible from single files
Run <code>graphify update .</code> after Python edits	Keep knowledge graph current (AST-only, free)

KNOWLEDGE GRAPH

Graphify — Codebase Knowledge Graph

`graphify-out/` contains an auto-generated knowledge graph of the entire codebase — a machine-readable map of every function, class, module, and the relationships between them.

File	Purpose
<code>graphify-out/GRAPH_REPORT.md</code>	Human-readable summary: god nodes, communities, key clusters
<code>graphify-out/graph.json</code>	Full graph data used by the <code>graphify</code> CLI
<code>graphify-out/cache/</code>	AST extraction cache — speeds up incremental updates

Current stats

7,400+ nodes · 12,000+ edges · 1,300+ communities

Commands

```
# Update graph after code changes (free, no LLM)
graphify update .

# Ask a natural-language question about the codebase
graphify query "how does the Celery task communicate with LangGraph?"

# Find the shortest relationship path between two modules
graphify path "document_processor" "extraction_agent"

# Explain a concept using the graph
graphify explain "PageIndex hierarchical traversal"
```

WORKFLOWS

Development Guide

Docker workflow (recommended)

```
# Start everything
docker compose up -d --build

# View logs
docker compose logs -f backend
docker compose logs -f worker

# Rebuild only backend/worker after tp_app or backend changes
docker compose up -d --build backend worker

# Shell into a container
docker compose exec backend bash
docker compose exec worker bash
```

Running backend locally

Postgres + Redis must already be running (e.g. `docker compose up -d db redis`).

```
cd backend
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver

# separate terminal:
celery -A config worker --loglevel=info
```

Running frontend locally

```
cd frontend
npm install
npm run dev

# vite.config.ts proxy →
#   target: http://localhost:8000
```

Testing AI agents without the UI

```
cd tp_app
uv run python -c "
from utils.dummy_data import DUMMY_STATE
from agents.orchestrator import run_agents
result = run_agents(DUMMY_STATE)
print(result.get('executive_summary', ''))
"
```

Adding a Python dependency to `tp_app`

```
cd tp_app
uv add <package-name>
# If backend also imports it, add to backend/requirements.txt manually
```

Git workflow

Always work on a feature branch — **never push directly to `main`**:

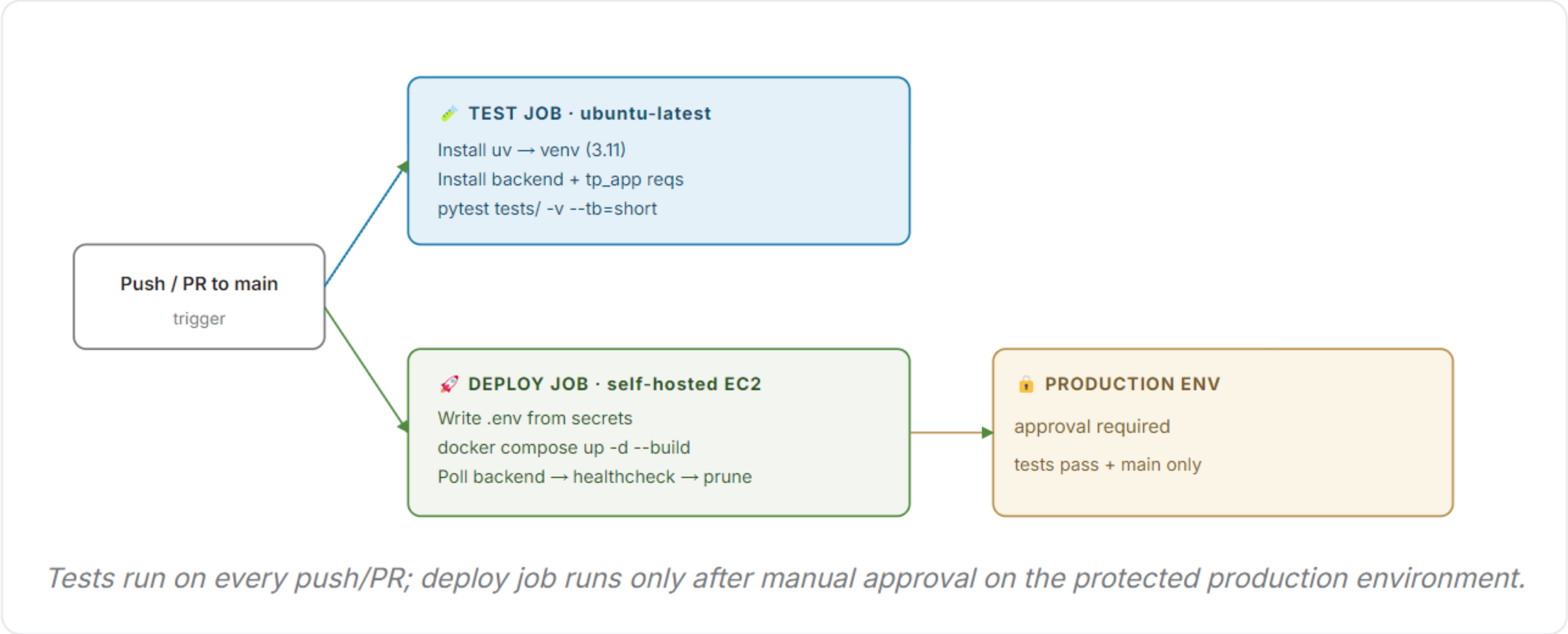
```
git checkout -b feature/your-feature-name
# make changes
git add <specific files>
git commit -m "feat: short description of what and why"
# Open a Pull Request to main via GitHub
```

PIPELINE

Deployment

CI/CD Pipeline

Defined in `.github/workflows/ci-cd.yml`, triggered on push to `main` or PR to `main`.



GitHub Secrets Setup

The pipeline writes `.env` directly from GitHub Secrets on every deploy.

Step 1 — Open Secrets in GitHub

1. Open the repository on GitHub
2. Click **Settings** (top right tab)
3. In the left sidebar, click **Secrets and variables** → **Actions**
4. Click **New repository secret** for each secret below

Step 2 — Add all required secrets

Secret	Example	Description
<code>SECRET_KEY</code>	<code>s3cr3t-random-50-chars ...</code>	Django secret key
<code>ALLOWED_HOSTS</code>	<code>yourdomain.com,www.yourdomain.com</code>	Production domain(s)
<code>POSTGRES_DB</code>	<code>tp_db</code>	Database name
<code>POSTGRES_USER</code>	<code>tp_user</code>	Database username
<code>POSTGRES_PASSWORD</code>	<code>str0ng-db-p4ssword!</code>	Strong DB password

Generate a secure `SECRET_KEY`:

```
python -c "import secrets; print(secrets.token_urlsafe(50))"
```

Step 3 — Create the production environment with protection rules

1. In the GitHub repository, click **Settings** → **Environments**
2. Click **New environment** → name it `production`
3. Enable **Required reviewers** → add your GitHub username
4. Click **Save protection rules**

Every push to `main` will wait for reviewer approval before deploying to EC2.

PRODUCTION

Production Server (EC2)

The application runs on an AWS EC2 instance managed by RSM. The self-hosted GitHub Actions runner is already installed and active on this server — no setup required.

Instance Details

Instance ID	i-0630a274e5494a46d
Name	RSM-TP-App
Instance type	t3.medium (2 vCPU, 4 GB)
Region	ap-southeast-3 (Jakarta)
Public IP	16.79.87.235 Elastic
Private IP	172.31.46.27
VPC	rsm-vpc
Subnet	rsm-private-default-subnet-2
Status	Running

Access URLs

Frontend	http://16.79.87.235:3000
Django API	http://16.79.87.235:8000/api/
Django Admin	http://16.79.87.235:8000/admin/

Public DNS

ec2-16-79-87-235.ap-southeast-3.compute.amazonaws.com

Useful commands on the server

```
# Check running containers
docker ps

# View live logs
docker compose logs -f backend
docker compose logs -f worker

# Restart all services
docker compose restart

# Rebuild and restart after a manual change
docker compose up -d --build

# Disk & memory
df -h
free -h
```

GitHub Actions Runner

The self-hosted runner is installed as a system service on the EC2 instance and starts automatically on reboot. To check its status:

```
# On the EC2 instance
sudo systemctl status actions.runner.*
```

If the runner shows as Offline

Go to GitHub → Settings → Actions → Runners. SSH into the instance and run:

```
sudo systemctl restart actions.runner.*
```

— BUILT WITH CARE BY —

From a small team in Jakarta, with *love*.

Weeks of late nights, way too much coffee, and a shared belief that consultants deserve better tools. This document — and the system it describes — exists because of the people in this photo.



RSM INDONESIA · TECHNOLOGY ADVISORY · APRIL 2026

Thank you for reading.

— The TP Local File Generator team